

Vector Calculator Server

Создано системой Doxygen 1.9.4

1	Алфавитный указатель пространств имен	1
1.1	Пространства имен	1
2	Алфавитный указатель классов	3
2.1	Классы	3
3	Список файлов	5
3.1	Файлы	5
4	Пространства имен	7
4.1	Пространство имен Config	7
4.1.1	Переменные	7
4.1.1.1	BUFFER_SIZE	7
4.1.1.2	DEFAULT_CLIENT_DB	8
4.1.1.3	DEFAULT_LOG_FILE	8
4.1.1.4	DEFAULT_PORT	8
4.1.1.5	ERR_MSG	8
4.1.1.6	MAX_LOGIN_LENGTH	8
4.1.1.7	MAX_PASSWORD_LENGTH	8
4.1.1.8	OK_MSG	9
4.1.1.9	SALT_BITS	9
4.1.1.10	SALT_HEX_LENGTH	9
5	Классы	11
5.1	Класс ClientDB	11
5.1.1	Подробное описание	12
5.1.2	Конструктор(ы)	12
5.1.2.1	ClientDB()	12
5.1.3	Методы	12
5.1.3.1	addClient()	12
5.1.3.2	clientExists()	13
5.1.3.3	generateHash()	13
5.1.3.4	generateSalt()	13
5.1.3.5	loadFromFile()	14
5.1.3.6	removeClient()	14
5.1.3.7	saveToFile()	14
5.1.3.8	verifyPassword()	15
5.1.4	Данные класса	16
5.1.4.1	clients	16
5.1.4.2	dbMutex	16
5.1.4.3	filename	16
5.2	Класс Logger	16
5.2.1	Подробное описание	17
5.2.2	Конструктор(ы)	17
5.2.2.1	Logger() [1/2]	17

5.2.2.2	<code>~Logger()</code>	18
5.2.2.3	<code>Logger()</code> [2/2]	18
5.2.3	Методы	18
5.2.3.1	<code>getCurrentTime()</code>	18
5.2.3.2	<code>initialize()</code>	18
5.2.3.3	<code>levelToString()</code>	18
5.2.3.4	<code>log()</code>	19
5.2.3.5	<code>logError()</code>	19
5.2.3.6	<code>operator=()</code>	19
5.2.4	Данные класса	20
5.2.4.1	<code>filename</code>	20
5.2.4.2	<code>logFile</code>	20
5.2.4.3	<code>logMutex</code>	20
5.3	Класс <code>Protocol</code>	20
5.3.1	Подробное описание	21
5.3.2	Методы	21
5.3.2.1	<code>binaryToHex()</code>	21
5.3.2.2	<code>hexToBinary()</code>	22
5.3.2.3	<code>receiveHash()</code>	22
5.3.2.4	<code>receiveLogin()</code>	23
5.3.2.5	<code>receiveVectorData()</code>	23
5.3.2.6	<code>recvAll()</code>	23
5.3.2.7	<code>sendAll()</code>	24
5.3.2.8	<code>sendError()</code>	24
5.3.2.9	<code>sendOk()</code>	25
5.3.2.10	<code>sendSalt()</code>	25
5.3.2.11	<code>sendVectorResults()</code>	25
5.3.3	Данные класса	26
5.3.3.1	<code>SEND_RECV_TIMEOUT</code>	26
5.4	Класс <code>Server</code>	26
5.4.1	Подробное описание	27
5.4.2	Конструктор(ы)	27
5.4.2.1	<code>Server()</code>	27
5.4.2.2	<code>~Server()</code>	28
5.4.3	Методы	28
5.4.3.1	<code>authenticateClient()</code>	28
5.4.3.2	<code>cleanup()</code>	28
5.4.3.3	<code>clientSession()</code>	29
5.4.3.4	<code>getConnectedClients()</code>	29
5.4.3.5	<code>handleClient()</code>	29
5.4.3.6	<code>initialize()</code>	29
5.4.3.7	<code>initializeSocket()</code>	30
5.4.3.8	<code>start()</code>	30

5.4.3.9 stop()	30
5.4.3.10 waitForStop()	30
5.4.4 Данные класса	30
5.4.4.1 clientDB	30
5.4.4.2 clientThreads	31
5.4.4.3 logger	31
5.4.4.4 port	31
5.4.4.5 running	31
5.4.4.6 serverSocket	31
5.5 Класс VectorProcessor	31
5.5.1 Подробное описание	32
5.5.2 Методы	32
5.5.2.1 calculateVectorSum()	32
5.5.2.2 processVectors()	33
5.5.2.3 readDouble()	33
5.5.2.4 readUInt32()	33
5.6 Структура VectorProcessor::VectorResult	34
5.6.1 Подробное описание	34
5.6.2 Данные класса	34
5.6.2.1 count	35
5.6.2.2 sums	35
6 Файлы	37
6.1 Файл include/ClientDB.h	37
6.1.1 Подробное описание	38
6.2 ClientDB.h	38
6.3 Файл include/Config.h	39
6.3.1 Подробное описание	40
6.4 Config.h	40
6.5 Файл include/Logger.h	41
6.5.1 Подробное описание	42
6.5.2 Перечисления	42
6.5.2.1 LogLevel	42
6.6 Logger.h	42
6.7 Файл include/Protocol.h	43
6.7.1 Подробное описание	44
6.8 Protocol.h	45
6.9 Файл include/Server.h	45
6.9.1 Подробное описание	46
6.10 Server.h	47
6.11 Файл include/VectorProcessor.h	47
6.11.1 Подробное описание	48
6.12 VectorProcessor.h	49

6.13 Файл <code>src/ClientDB.cpp</code>	49
6.14 Файл <code>src/Logger.cpp</code>	49
6.15 Файл <code>src/main.cpp</code>	50
6.15.1 Функции	50
6.15.1.1 <code>main()</code>	51
6.15.1.2 <code>printHelp()</code>	51
6.15.1.3 <code>signalHandler()</code>	51
6.15.2 Переменные	51
6.15.2.1 <code>server</code>	51
6.16 Файл <code>src/Protocol.cpp</code>	51
6.17 Файл <code>src/Server.cpp</code>	52
6.18 Файл <code>src/VectorProcessor.cpp</code>	52
Предметный указатель	53

Глава 1

Алфавитный указатель пространств имен

1.1 Пространства имен

Полный список пространств имен.

Config	7
------------------------	---

Глава 2

Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

ClientDB	Класс для управления базой данных клиентов	11
Logger	Класс для ведения журнала событий	16
Protocol	Класс для работы с сетевым протоколом	20
Server	Основной класс сервера векторного калькулятора	26
VectorProcessor	Класс для обработки векторных данных	31
VectorProcessor::VectorResult	Структура результата обработки векторов	34

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

include/ ClientDB.h	
Класс для работы с базой данных клиентов	37
include/ Config.h	
Заголовочный файл с конфигурационными константами сервера	39
include/ Logger.h	
Класс для логирования событий сервера	41
include/ Protocol.h	
Класс для работы с сетевым протоколом сервера	43
include/ Server.h	
Основной класс сервера векторного калькулятора	45
include/ VectorProcessor.h	
Класс для обработки векторных данных	47
src/ ClientDB.cpp	49
src/ Logger.cpp	49
src/ main.cpp	50
src/ Protocol.cpp	51
src/ Server.cpp	52
src/ VectorProcessor.cpp	52

Глава 4

Пространства имен

4.1 Пространство имен Config

Переменные

- `const std::string DEFAULT_CLIENT_DB = "/etc/vcalc.conf"`
Файл базы клиентов по умолчанию
- `const std::string DEFAULT_LOG_FILE = "/var/log/vcalc.log"`
Файл журнала по умолчанию
- `const int DEFAULT_PORT = 33333`
Порт сервера по умолчанию
- `const int SALT_BITS = 64`
Размер соли в битах
- `const int SALT_HEX_LENGTH = 16`
Длина соли в hex символах
- `const int MAX_LOGIN_LENGTH = 32`
Максимальная длина логина
- `const int MAX_PASSWORD_LENGTH = 256`
Максимальная длина пароля
- `const int BUFFER_SIZE = 4096`
Размер буфера для сетевых операций
- `const std::string ERR_MSG = "ERR"`
Сообщение об ошибке
- `const std::string OK_MSG = "OK"`
Сообщение об успехе

4.1.1 Переменные

4.1.1.1 BUFFER_SIZE

```
const int Config::BUFFER_SIZE = 4096
```

Размер буфера для сетевых операций

4.1.1.2 DEFAULT_CLIENT_DB

```
const std::string Config::DEFAULT_CLIENT_DB = "/etc/vcalc.conf"
```

Файл базы клиентов по умолчанию

4.1.1.3 DEFAULT_LOG_FILE

```
const std::string Config::DEFAULT_LOG_FILE = "/var/log/vcalc.log"
```

Файл журнала по умолчанию

4.1.1.4 DEFAULT_PORT

```
const int Config::DEFAULT_PORT = 33333
```

Порт сервера по умолчанию

4.1.1.5 ERR_MSG

```
const std::string Config::ERR_MSG = "ERR"
```

Сообщение об ошибке

4.1.1.6 MAX_LOGIN_LENGTH

```
const int Config::MAX_LOGIN_LENGTH = 32
```

Максимальная длина логина

4.1.1.7 MAX_PASSWORD_LENGTH

```
const int Config::MAX_PASSWORD_LENGTH = 256
```

Максимальная длина пароля

4.1.1.8 OK_MSG

```
const std::string Config::OK_MSG = "OK"
```

Сообщение об успехе

4.1.1.9 SALT_BITS

```
const int Config::SALT_BITS = 64
```

Размер соли в битах

4.1.1.10 SALT_HEX_LENGTH

```
const int Config::SALT_HEX_LENGTH = 16
```

Длина соли в hex символах

Глава 5

Классы

5.1 Класс ClientDB

Класс для управления базой данных клиентов

```
#include <ClientDB.h>
```

Открытые члены

- `ClientDB` (const std::string &filename)
Конструктор класса `ClientDB`.
- bool `loadFromFile` ()
Загрузка клиентов из файла
- bool `saveToFile` ()
Сохранение клиентов в файл
- bool `clientExists` (const std::string &login) const
Проверка существования клиента
- bool `verifyPassword` (const std::string &login, const std::string &passwordHash) const
Проверка хэша пароля клиента
- bool `addClient` (const std::string &login, const std::string &password)
Добавление нового клиента
- bool `removeClient` (const std::string &login)
Удаление клиента

Открытые статические члены

- static std::string `generateHash` (const std::string &salt, const std::string &password)
Генерация хэша SHA-1 для проверки пароля
- static std::string `generateSalt` ()
Генерация случайной соли

Закрытые данные

- `std::unordered_map< std::string, std::string >` `clients`
Хранилище клиентов: логин -> хэш пароля
- `std::mutex` `dbMutex`
Мьютекс для потокобезопасности (mutable для const методов)
- `std::string` `filename`
Имя файла базы данных

5.1.1 Подробное описание

Класс для управления базой данных клиентов

Реализует хранение пар логин-хэш_пароля в памяти с возможностью сохранения в файл и загрузки из него. Потокобезопасен за счет использования мьютекса.

5.1.2 Конструктор(ы)

5.1.2.1 ClientDB()

```
ClientDB::ClientDB (
    const std::string & filename )
```

Конструктор класса `ClientDB`.

Аргументы

in	filename	Имя файла для хранения базы клиентов
----	----------	--------------------------------------

5.1.3 Методы

5.1.3.1 addClient()

```
bool ClientDB::addClient (
    const std::string & login,
    const std::string & password )
```

Добавление нового клиента

Аргументы

in	login	Логин нового клиента
in	password	Пароль нового клиента

Возвращает

true если клиент успешно добавлен, false в случае ошибки

5.1.3.2 clientExists()

```
bool ClientDB::clientExists (
    const std::string & login ) const
```

Проверка существования клиента

Аргументы

in	login	Логин клиента для проверки
----	-------	----------------------------

Возвращает

true если клиент существует, false в противном случае

5.1.3.3 generateHash()

```
std::string ClientDB::generateHash (
    const std::string & salt,
    const std::string & password ) [static]
```

Генерация хэша SHA-1 для проверки пароля

Аргументы

in	salt	Соль для хэширования
in	password	Пароль для хэширования

Возвращает

Хэш SHA-1 в виде hex строки в верхнем регистре

5.1.3.4 generateSalt()

```
std::string ClientDB::generateSalt ( ) [static]
```

Генерация случайной соли

Возвращает

Случайная соль длиной 16 hex символов в верхнем регистре

5.1.3.5 loadFromFile()

```
bool ClientDB::loadFromFile ( )
```

Загрузка клиентов из файла

Возвращает

true если файл успешно загружен, false в случае ошибки

5.1.3.6 removeClient()

```
bool ClientDB::removeClient (
    const std::string & login )
```

Удаление клиента

Аргументы

in	login	Логин клиента для удаления
----	-------	----------------------------

Возвращает

true если клиент успешно удален, false если клиент не найден

5.1.3.7 saveToFile()

```
bool ClientDB::saveToFile ( )
```

Сохранение клиентов в файл

Возвращает

true если файл успешно сохранен, false в случае ошибки

5.1.3.8 verifyPassword()

```
bool ClientDB::verifyPassword (
    const std::string & login,
    const std::string & passwordHash ) const
```

Проверка хэша пароля клиента

Аргументы

in	login	Логин клиента
in	passwordHash	Хэш пароля для проверки

Возвращает

true если хэш совпадает, false в противном случае

5.1.4 Данные класса

5.1.4.1 clients

```
std::unordered_map<std::string, std::string> ClientDB::clients [private]
```

Хранилище клиентов: логин -> хэш пароля

5.1.4.2 dbMutex

```
std::mutex ClientDB::dbMutex [mutable], [private]
```

Мьютекс для потокобезопасности (mutable для const методов)

5.1.4.3 filename

```
std::string ClientDB::filename [private]
```

Имя файла базы данных

Объявления и описания членов классов находятся в файлах:

- `include/ClientDB.h`
- `src/ClientDB.cpp`

5.2 Класс Logger

Класс для ведения журнала событий

```
#include <Logger.h>
```

Открытые члены

- `Logger` (`const std::string &filename`)
Конструктор класса `Logger`.
- `~Logger` ()
Деструктор класса `Logger`.
- `bool initialize` ()
Инициализация логгера
- `void log` (`LogLevel level`, `const std::string &message`, `const std::string ¶ms=""`)
Запись сообщения в лог
- `void logError` (`bool isCritical`, `const std::string &message`, `const std::string ¶ms=""`)
Запись ошибки в лог
- `Logger` (`const Logger &`)=delete
- `Logger & operator=` (`const Logger &`)=delete

Закрытые члены

- `std::string getCurrentTime` () const
Получение текущего времени в формате строки
- `std::string levelToString` (`LogLevel level`) const
Преобразование уровня логирования в строку

Закрытые данные

- `std::ofstream logFile`
Файл для записи логов
- `std::mutex logMutex`
Мьютекс для потокобезопасности
- `std::string filename`
Имя файла журнала

5.2.1 Подробное описание

Класс для ведения журнала событий

Реализует потокобезопасное логирование в файл с поддержкой различных уровней важности сообщений.

5.2.2 Конструктор(ы)

5.2.2.1 `Logger()` [1/2]

```
Logger::Logger (  
    const std::string & filename )
```

Конструктор класса `Logger`.

Аргументы

in	filename	Имя файла для записи логов
----	----------	----------------------------

5.2.2.2 ~Logger()

Logger::~Logger ()

Деструктор класса [Logger](#).

Автоматически закрывает файл логов

5.2.2.3 Logger() [2/2]

Logger::Logger (
 const [Logger](#) &) [delete]

5.2.3 Методы

5.2.3.1 getCurrentTime()

std::string Logger::getCurrentTime () const [private]

Получение текущего времени в формате строки

Возвращает

Строка с текущим временем в формате "ГГГГ-ММ-ДД ЧЧ:ММ:СС"

5.2.3.2 initialize()

bool Logger::initialize ()

Инициализация логгера

Возвращает

true если файл успешно открыт, false в случае ошибки

5.2.3.3 levelToString()

std::string Logger::levelToString (
 [LogLevel](#) level) const [private]

Преобразование уровня логирования в строку

Аргументы

in	level	Уровень логирования
----	-------	---------------------

Возвращает

Строковое представление уровня

5.2.3.4 log()

```
void Logger::log (
    LogLevel level,
    const std::string & message,
    const std::string & params = "" )
```

Запись сообщения в лог

Аргументы

in	level	Уровень важности сообщения
in	message	Текст сообщения
in	params	Дополнительные параметры (опционально)

5.2.3.5 logError()

```
void Logger::logError (
    bool isCritical,
    const std::string & message,
    const std::string & params = "" )
```

Запись ошибки в лог

Аргументы

in	isCritical	Признак критичности ошибки
in	message	Текст сообщения об ошибке
in	params	Дополнительные параметры (опционально)

5.2.3.6 operator=()

```
Logger & Logger::operator= (
    const Logger & ) [delete]
```

5.2.4 Данные класса

5.2.4.1 filename

`std::string Logger::filename [private]`

Имя файла журнала

5.2.4.2 logFile

`std::ofstream Logger::logFile [private]`

Файл для записи логов

5.2.4.3 logMutex

`std::mutex Logger::logMutex [mutable], [private]`

Мьютекс для потокобезопасности

Объявления и описания членов классов находятся в файлах:

- `include/Logger.h`
- `src/Logger.cpp`

5.3 Класс Protocol

Класс для работы с сетевым протоколом

```
#include <Protocol.h>
```

Открытые статические члены

- static bool `sendSalt` (int clientSocket, const std::string &salt)
Отправка соли клиенту
- static bool `sendError` (int clientSocket)
Отправка сообщения об ошибке
- static bool `sendOk` (int clientSocket)
Отправка сообщения об успехе
- static bool `receiveLogin` (int clientSocket, std::string &login)
Получение логина от клиента
- static bool `receiveHash` (int clientSocket, std::string &hash)
Получение хэша от клиента
- static bool `sendVectorResults` (int clientSocket, const std::vector< double > &results)
Отправка результатов вычисления векторов
- static bool `receiveVectorData` (int clientSocket, std::vector< uint8_t > &binaryData)
Получение векторных данных от клиента
- static bool `sendAll` (int socket, const void *buffer, size_t length)
Отправка всех данных через сокет
- static bool `recvAll` (int socket, void *buffer, size_t length)
Получение всех данных через сокет
- static std::string `binaryToHex` (const std::vector< uint8_t > &data)
Преобразование бинарных данных в hex строку
- static std::vector< uint8_t > `hexToBinary` (const std::string &hex)
Преобразование hex строки в бинарные данные

Закрытые статические данные

- static const int `SEND_RECV_TIMEOUT` = 10
Таймаут отправки/приема в секундах

5.3.1 Подробное описание

Класс для работы с сетевым протоколом

Содержит статические методы для отправки и приема данных по сетевому протоколу векторного калькулятора.

5.3.2 Методы

5.3.2.1 `binaryToHex()`

```
static std::string Protocol::binaryToHex (
    const std::vector< uint8_t > & data ) [static]
```

Преобразование бинарных данных в hex строку

Аргументы

in	data	Бинарные данные
----	------	-----------------

Возвращает

Нех строка в верхнем регистре

5.3.2.2 hexToBinary()

```
static std::vector< uint8_t > Protocol::hexToBinary (
    const std::string & hex ) [static]
```

Преобразование hex строки в бинарные данные

Аргументы

in	hex	Нех строка
----	-----	------------

Возвращает

Бинарные данные

5.3.2.3 receiveHash()

```
bool Protocol::receiveHash (
    int clientSocket,
    std::string & hash ) [static]
```

Получение хэша от клиента

Аргументы

in	clientSocket	Сокет клиента
out	hash	Полученный хэш

Возвращает

true если хэш успешно получен, false в случае ошибки

5.3.2.4 receiveLogin()

```
bool Protocol::receiveLogin (
    int clientSocket,
    std::string & login ) [static]
```

Получение логина от клиента

Аргументы

in	clientSocket	Сокет клиента
out	login	Полученный логин

Возвращает

true если логин успешно получен, false в случае ошибки

5.3.2.5 receiveVectorData()

```
bool Protocol::receiveVectorData (
    int clientSocket,
    std::vector< uint8_t > & binaryData ) [static]
```

Получение векторных данных от клиента

Аргументы

in	clientSocket	Сокет клиента
out	binaryData	Полученные бинарные данные

Возвращает

true если данные успешно получены, false в случае ошибки

5.3.2.6 recvAll()

```
bool Protocol::recvAll (
    int socket,
    void * buffer,
    size_t length ) [static]
```

Получение всех данных через сокет

Аргументы

in	socket	Сокет для приема
out	buffer	Буфер для данных
in	length	Ожидаемая длина данных

Возвращает

true если все данные получены, false в случае ошибки

5.3.2.7 sendAll()

```
bool Protocol::sendAll (
    int socket,
    const void * buffer,
    size_t length ) [static]
```

Отправка всех данных через сокет

Аргументы

in	socket	Сокет для отправки
in	buffer	Буфер с данными
in	length	Длина данных

Возвращает

true если все данные отправлены, false в случае ошибки

5.3.2.8 sendError()

```
bool Protocol::sendError (
    int clientSocket ) [static]
```

Отправка сообщения об ошибке

Аргументы

in	clientSocket	Сокет клиента
----	--------------	---------------

Возвращает

true если сообщение успешно отправлено, false в случае ошибки

5.3.2.9 sendOk()

```
bool Protocol::sendOk (
    int clientSocket ) [static]
```

Отправка сообщения об успехе

Аргументы

in	clientSocket	Сокет клиента
----	--------------	---------------

Возвращает

true если сообщение успешно отправлено, false в случае ошибки

5.3.2.10 sendSalt()

```
bool Protocol::sendSalt (
    int clientSocket,
    const std::string & salt ) [static]
```

Отправка соли клиенту

Аргументы

in	clientSocket	Сокет клиента
in	salt	Соль для отправки

Возвращает

true если соль успешно отправлена, false в случае ошибки

5.3.2.11 sendVectorResults()

```
bool Protocol::sendVectorResults (
    int clientSocket,
    const std::vector< double > & results ) [static]
```

Отправка результатов вычисления векторов

Аргументы

in	clientSocket	Сокет клиента
in	results	Результаты вычислений

Возвращает

true если результаты успешно отправлены, false в случае ошибки

5.3.3 Данные класса

5.3.3.1 SEND_RECV_TIMEOUT

```
const int Protocol::SEND_RECV_TIMEOUT = 10 [static], [private]
```

Таймаут отправки/приема в секундах

Объявления и описания членов классов находятся в файлах:

- include/[Protocol.h](#)
- src/[Protocol.cpp](#)

5.4 Класс Server

Основной класс сервера векторного калькулятора

```
#include <Server.h>
```

Открытые члены

- [Server](#) (int [port](#), const std::string &clientDbFile, const std::string &logFile)
Конструктор класса [Server](#).
- [~Server](#) ()
Деструктор класса [Server](#).
- bool [initialize](#) ()
Инициализация сервера
- void [start](#) ()
Запуск сервера
- void [stop](#) ()
Остановка сервера
- void [waitForStop](#) ()
Ожидание остановки сервера
- size_t [getConnectedClients](#) () const
Получение количества подключенных клиентов

Закрытые члены

- bool `initializeSocket` ()
Инициализация сетевого сокета
- void `cleanup` ()
Очистка ресурсов сервера
- void `handleClient` (int clientSocket)
Обработка подключения клиента
- void `clientSession` (int clientSocket)
Сессия работы с клиентом
- bool `authenticateClient` (int clientSocket, std::string &clientLogin)
Аутентификация клиента

Закрытые данные

- int `port`
Порт сервера
- int `serverSocket`
Основной сокет сервера
- std::atomic< bool > `running`
Флаг работы сервера (atomic для потокобезопасности)
- std::unique_ptr< `Logger` > `logger`
Логгер сервера
- std::unique_ptr< `ClientDB` > `clientDB`
База данных клиентов
- std::vector< std::thread > `clientThreads`
Потоки обработки клиентов

5.4.1 Подробное описание

Основной класс сервера векторного калькулятора

Управляет сетевыми подключениями, аутентификацией клиентов, обработкой векторных данных и многопоточностью.

5.4.2 Конструктор(ы)

5.4.2.1 Server()

```
Server::Server (  
    int port,  
    const std::string & clientDbFile,  
    const std::string & logFile )
```

Конструктор класса `Server`.

Аргументы

in	port	Порт для прослушивания
in	clientDbFile	Файл базы данных клиентов
in	logFile	Файл журнала

5.4.2.2 ~Server()

Server::~~Server ()

Деструктор класса [Server](#).

Автоматически останавливает сервер и очищает ресурсы

5.4.3 Методы

5.4.3.1 authenticateClient()

```
bool Server::authenticateClient (
    int clientSocket,
    std::string & clientLogin ) [private]
```

Аутентификация клиента

Аргументы

in	clientSocket	Сокет клиента
out	clientLogin	Логин аутентифицированного клиента

Возвращает

true если аутентификация успешна, false в противном случае

5.4.3.2 cleanup()

```
void Server::cleanup ( ) [private]
```

Очистка ресурсов сервера

5.4.3.3 clientSession()

```
void Server::clientSession (
    int clientSocket ) [private]
```

Сессия работы с клиентом

Аргументы

in	clientSocket	Сокет клиента
----	--------------	---------------

5.4.3.4 getConnectedClients()

```
size_t Server::getConnectedClients ( ) const
```

Получение количества подключенных клиентов

Возвращает

Количество активных клиентских потоков

5.4.3.5 handleClient()

```
void Server::handleClient (
    int clientSocket ) [private]
```

Обработка подключения клиента

Аргументы

in	clientSocket	Сокет клиента
----	--------------	---------------

5.4.3.6 initialize()

```
bool Server::initialize ( )
```

Инициализация сервера

Возвращает

true если инициализация успешна, false в случае ошибки

5.4.3.7 initializeSocket()

```
bool Server::initializeSocket ( ) [private]
```

Инициализация сетевого сокета

Возвращает

true если сокет успешно инициализирован, false в случае ошибки

5.4.3.8 start()

```
void Server::start ( )
```

Запуск сервера

Начинает прослушивание порта и обработку подключений

5.4.3.9 stop()

```
void Server::stop ( )
```

Остановка сервера

Graceful shutdown сервера

5.4.3.10 waitForStop()

```
void Server::waitForStop ( )
```

Ожидание остановки сервера

Ожидает завершения всех клиентских потоков

5.4.4 Данные класса

5.4.4.1 clientDB

```
std::unique_ptr<ClientDB> Server::clientDB [private]
```

База данных клиентов

5.4.4.2 clientThreads

```
std::vector<std::thread> Server::clientThreads [private]
```

Потоки обработки клиентов

5.4.4.3 logger

```
std::unique_ptr<Logger> Server::logger [private]
```

Логгер сервера

5.4.4.4 port

```
int Server::port [private]
```

Порт сервера

5.4.4.5 running

```
std::atomic<bool> Server::running [private]
```

Флаг работы сервера (atomic для потокобезопасности)

5.4.4.6 serverSocket

```
int Server::serverSocket [private]
```

Основной сокет сервера

Объявления и описания членов классов находятся в файлах:

- include/[Server.h](#)
- src/[Server.cpp](#)

5.5 Класс VectorProcessor

Класс для обработки векторных данных

```
#include <VectorProcessor.h>
```

Классы

- struct [VectorResult](#)
Структура результата обработки векторов

Открытые статические члены

- static [VectorResult processVectors](#) (const std::vector< uint8_t > &binaryData)
Обработка бинарных векторных данных
- static double [calculateVectorSum](#) (const std::vector< double > &vector)
Вычисление суммы элементов вектора

Закрытые статические члены

- static bool [readUInt32](#) (const uint8_t *data, size_t &offset, size_t maxSize, uint32_t &value)
Чтение 32-битного целого числа из бинарных данных
- static bool [readDouble](#) (const uint8_t *data, size_t &offset, size_t maxSize, double &value)
Чтение числа double из бинарных данных

5.5.1 Подробное описание

Класс для обработки векторных данных

Реализует парсинг бинарных данных векторов и вычисление суммы их элементов с использованием алгоритма Кэхэна для уменьшения ошибок округления.

5.5.2 Методы

5.5.2.1 calculateVectorSum()

```
double VectorProcessor::calculateVectorSum (
    const std::vector< double > & vector ) [static]
```

Вычисление суммы элементов вектора

Аргументы

in	vector	Вектор для вычисления суммы
----	--------	-----------------------------

Возвращает

Сумма элементов вектора

Заметки

Использует алгоритм Кэхэна для уменьшения ошибок округления

5.5.2.2 processVectors()

```
VectorProcessor::VectorResult VectorProcessor::processVectors (  
    const std::vector< uint8_t > & binaryData )    [static]
```

Обработка бинарных векторных данных

Аргументы

in	binaryData	Бинарные данные векторов
----	------------	--------------------------

Возвращает

Структура с результатами обработки

5.5.2.3 readDouble()

```
bool VectorProcessor::readDouble (  
    const uint8_t * data,  
    size_t & offset,  
    size_t maxSize,  
    double & value )    [static], [private]
```

Чтение числа double из бинарных данных

Аргументы

in	data	Указатель на бинарные данные
in,out	offset	Текущая позиция в данных (смещается при чтении)
in	maxSize	Максимальный размер данных
out	value	Прочитанное значение

Возвращает

true если чтение успешно, false в случае ошибки

5.5.2.4 readUInt32()

```
bool VectorProcessor::readUInt32 (  
    const uint8_t * data,
```

```
size_t & offset,
size_t maxSize,
uint32_t & value ) [static], [private]
```

Чтение 32-битного целого числа из бинарных данных

Аргументы

in	data	Указатель на бинарные данные
in,out	offset	Текущая позиция в данных (смещается при чтении)
in	maxSize	Максимальный размер данных
out	value	Прочитанное значение

Возвращает

true если чтение успешно, false в случае ошибки

Объявления и описания членов классов находятся в файлах:

- [include/VectorProcessor.h](#)
- [src/VectorProcessor.cpp](#)

5.6 Структура VectorProcessor::VectorResult

Структура результата обработки векторов

```
#include <VectorProcessor.h>
```

Открытые атрибуты

- `uint32_t` [count](#)
Количество обработанных векторов
- `std::vector< double >` [sums](#)
Суммы элементов каждого вектора

5.6.1 Подробное описание

Структура результата обработки векторов

5.6.2 Данные класса

5.6.2.1 `count`

```
uint32_t VectorProcessor::VectorResult::count
```

Количество обработанных векторов

5.6.2.2 `sums`

```
std::vector<double> VectorProcessor::VectorResult::sums
```

Суммы элементов каждого вектора

Объявления и описания членов структуры находятся в файле:

- `include/VectorProcessor.h`

Глава 6

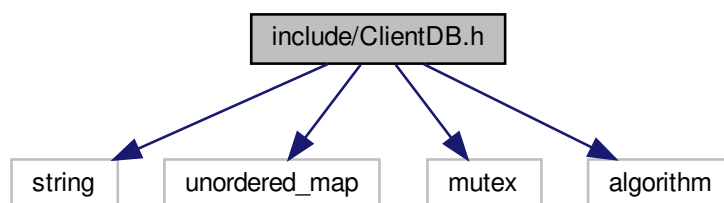
Файлы

6.1 Файл include/ClientDB.h

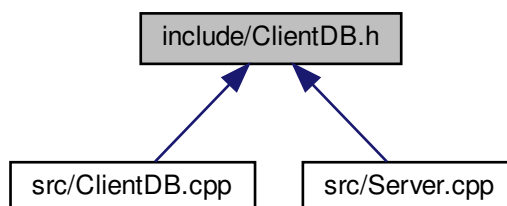
Класс для работы с базой данных клиентов

```
#include <string>
#include <unordered_map>
#include <mutex>
#include <algorithm>
```

Граф включаемых заголовочных файлов для ClientDB.h:



Граф файлов, в которые включается этот файл:



Классы

- class `ClientDB`

Класс для управления базой данных клиентов

6.1.1 Подробное описание

Класс для работы с базой данных клиентов

Автор

Alexander Melekhin

Версия

1.0

Дата

2024

Обеспечивает хранение, загрузку, сохранение и проверку учетных данных клиентов. Использует SHA-1 для хэширования паролей.

Предупреждения

Потокобезопасный доступ к базе клиентов

6.2 ClientDB.h

[См. документацию.](#)

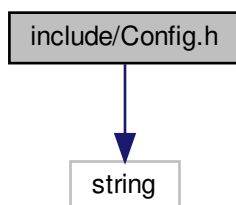
```
1
13 #ifndef CLIENTDB_H
14 #define CLIENTDB_H
15
16 #include <string>
17 #include <unordered_map>
18 #include <mutex>
19 #include <algorithm>
20
28 class ClientDB {
29 private:
30     std::unordered_map<std::string, std::string> clients;
31     mutable std::mutex dbMutex;
32     std::string filename;
33
34 public:
39     ClientDB(const std::string& filename);
40
45     bool loadFromFile();
46
51     bool saveToFile();
52
58     bool clientExists(const std::string& login) const;
59
66     bool verifyPassword(const std::string& login, const std::string& passwordHash) const;
67
74     static std::string generateHash(const std::string& salt, const std::string& password);
75
80     static std::string generateSalt();
81
88     bool addClient(const std::string& login, const std::string& password);
89
95     bool removeClient(const std::string& login);
96 };
97
98 #endif // CLIENTDB_H
```

6.3 Файл include/Config.h

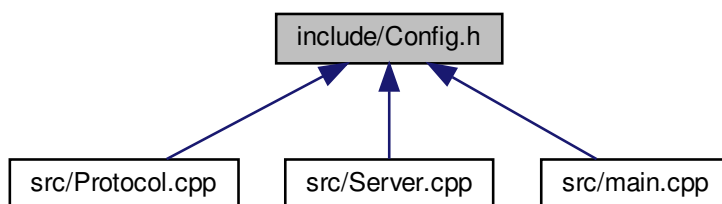
Заголовочный файл с конфигурационными константами сервера

```
#include <string>
```

Граф включаемых заголовочных файлов для Config.h:



Граф файлов, в которые включается этот файл:



Пространства имен

- namespace `Config`

Переменные

- `const std::string Config::DEFAULT_CLIENT_DB = "/etc/vcalc.conf"`
Файл базы клиентов по умолчанию
- `const std::string Config::DEFAULT_LOG_FILE = "/var/log/vcalc.log"`
Файл журнала по умолчанию
- `const int Config::DEFAULT_PORT = 33333`
Порт сервера по умолчанию
- `const int Config::SALT_BITS = 64`
Размер соли в битах

- `const int Config::SALT_HEX_LENGTH = 16`
Длина соли в hex символах
- `const int Config::MAX_LOGIN_LENGTH = 32`
Максимальная длина логина
- `const int Config::MAX_PASSWORD_LENGTH = 256`
Максимальная длина пароля
- `const int Config::BUFFER_SIZE = 4096`
Размер буфера для сетевых операций
- `const std::string Config::ERR_MSG = "ERR"`
Сообщение об ошибке
- `const std::string Config::OK_MSG = "OK"`
Сообщение об успехе

6.3.1 Подробное описание

Заголовочный файл с конфигурационными константами сервера

Автор

Alexander Melekhin

Версия

1.0

Дата

2024

Содержит константы по умолчанию для порта, файлов, размеров данных и сообщений протокола

Предупреждения

Изменение констант может повлиять на совместимость с клиентами

6.4 Config.h

[См. документацию.](#)

```

1
13 #ifndef CONFIG_H
14 #define CONFIG_H
15
16 #include <string>
17
18 namespace Config {
19     // Константы по умолчанию
20     const std::string DEFAULT_CLIENT_DB = "/etc/vcalc.conf";
21     const std::string DEFAULT_LOG_FILE = "/var/log/vcalc.log";
22     const int DEFAULT_PORT = 33333;
23
24     // Размеры данных
25     const int SALT_BITS = 64;
26     const int SALT_HEX_LENGTH = 16;
27     const int MAX_LOGIN_LENGTH = 32;
28     const int MAX_PASSWORD_LENGTH = 256;
29
30     // Протокол
31     const int BUFFER_SIZE = 4096;
32     const std::string ERR_MSG = "ERR";
33     const std::string OK_MSG = "OK";
34 }
35
36 #endif // CONFIG_H

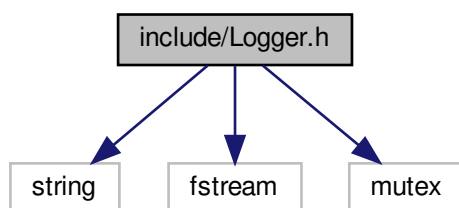
```

6.5 Файл include/Logger.h

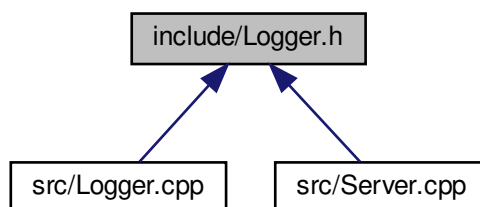
Класс для логирования событий сервера

```
#include <string>
#include <fstream>
#include <mutex>
```

Граф включаемых заголовочных файлов для Logger.h:



Граф файлов, в которые включается этот файл:



Классы

- class `Logger`

Класс для ведения журнала событий

Перечисления

- enum class `LogLevel` { `INFO` , `WARNING` , `ERROR` , `CRITICAL` }

Уровни логирования

6.5.1 Подробное описание

Класс для логирования событий сервера

Автор

Alexander Melekhin

Версия

1.0

Дата

2024

Обеспечивает запись логов в файл с различными уровнями важности. Потокбезопасный. Автоматически закрывает файл при уничтожении.

6.5.2 Перечисления

6.5.2.1 LogLevel

enum class `LogLevel` [strong]

Уровни логирования

Элементы перечислений

INFO	Информационные сообщения
WARNING	Предупреждения
ERROR	Ошибки
CRITICAL	Критические ошибки

6.6 Logger.h

[См. документацию.](#)

```
1
12 #ifndef LOGGER_H
13 #define LOGGER_H
14
15 #include <string>
16 #include <fstream>
17 #include <mutex>
18
22 enum class LogLevel {
23     INFO,
```

```

24  WARNING,
25  ERROR,
26  CRITICAL
27 };
28
29 class Logger {
30 private:
31     std::ofstream logFile;
32     mutable std::mutex logMutex;
33     std::string filename;
34
35     std::string getCurrentTime() const;
36
37     std::string levelToString(LogLevel level) const;
38
39 public:
40     Logger(const std::string& filename);
41
42     ~Logger();
43
44     bool initialize();
45
46     void log(LogLevel level, const std::string& message, const std::string& params = "");
47
48     void logError(bool isCritical, const std::string& message, const std::string& params = "");
49
50     // Запрет копирования
51     Logger(const Logger&) = delete;
52     Logger& operator=(const Logger&) = delete;
53 };
54 #endif // LOGGER_H

```

6.7 Файл include/Protocol.h

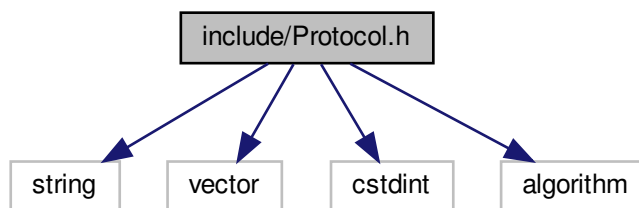
Класс для работы с сетевым протоколом сервера

```

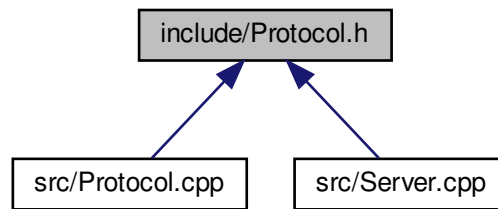
#include <string>
#include <vector>
#include <cstdint>
#include <algorithm>

```

Граф включаемых заголовочных файлов для Protocol.h:



Граф файлов, в которые включается этот файл:



Классы

- class `Protocol`

Класс для работы с сетевым протоколом

6.7.1 Подробное описание

Класс для работы с сетевым протоколом сервера

Автор

Alexander Melekhin

Версия

1.0

Дата

2024

Реализует бинарный протокол обмена данными между сервером и клиентами, включая аутентификацию и передачу векторных данных.

6.8 Protocol.h

[См. документацию.](#)

```

1
12 #ifndef PROTOCOL_H
13 #define PROTOCOL_H
14
15 #include <string>
16 #include <vector>
17 #include <stdint>
18 #include <algorithm>
19
20 class Protocol {
21 public:
22     // Аутентификация
23     static bool sendSalt(int clientSocket, const std::string& salt);
24
25     static bool sendError(int clientSocket);
26
27     static bool sendOk(int clientSocket);
28
29     static bool receiveLogin(int clientSocket, std::string& login);
30
31     static bool receiveHash(int clientSocket, std::string& hash);
32
33     // Работа с векторами (бинарный формат)
34     static bool sendVectorResults(int clientSocket,
35                                   const std::vector<double>& results);
36
37     static bool receiveVectorData(int clientSocket,
38                                   std::vector<uint8_t>& binaryData);
39
40     // Вспомогательные функции
41     static bool sendAll(int socket, const void* buffer, size_t length);
42
43     static bool recvAll(int socket, void* buffer, size_t length);
44
45     static std::string binaryToHex(const std::vector<uint8_t>& data);
46
47     static std::vector<uint8_t> hexToBinary(const std::string& hex);
48
49 private:
50     static const int SEND_RECV_TIMEOUT = 10;
51 };
52 #endif // PROTOCOL_H

```

6.9 Файл include/Server.h

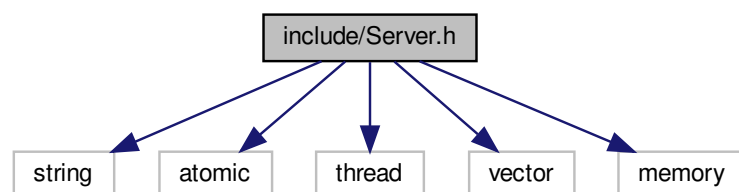
Основной класс сервера векторного калькулятора

```

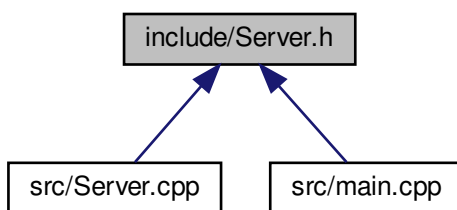
#include <string>
#include <atomic>
#include <thread>
#include <vector>
#include <memory>

```

Граф включаемых заголовочных файлов для Server.h:



Граф файлов, в которые включается этот файл:



Классы

- class [Server](#)

Основной класс сервера векторного калькулятора

6.9.1 Подробное описание

Основной класс сервера векторного калькулятора

Автор

Alexander Melekhin

Версия

1.0

Дата

2024

Реализует многопоточный TCP сервер с аутентификацией клиентов и обработкой векторных вычислений.

6.10 Server.h

См. документацию.

```

1
12 #ifndef SERVER_H
13 #define SERVER_H
14
15 #include <string>
16 #include <atomic>
17 #include <thread>
18 #include <vector>
19 #include <memory>
20
21 // Предварительные объявления
22 class Logger;
23 class ClientDB;
24
25 class Server {
26 private:
27     int port;
28     int serverSocket;
29     std::atomic<bool> running;
30     std::unique_ptr<Logger> logger;
31     std::unique_ptr<ClientDB> clientDB;
32
33     std::vector<std::thread> clientThreads;
34
35     bool initializeSocket();
36
37     void cleanup();
38
39     void handleClient(int clientSocket);
40
41     void clientSession(int clientSocket);
42
43     bool authenticateClient(int clientSocket, std::string& clientLogin);
44
45 public:
46     Server(int port, const std::string& clientDbFile, const std::string& logFile);
47
48     ~Server();
49
50     bool initialize();
51
52     void start();
53
54     void stop();
55
56     void waitForStop();
57
58     size_t getConnectedClients() const;
59 };
60
61 #endif // SERVER_H

```

6.11 Файл include/VectorProcessor.h

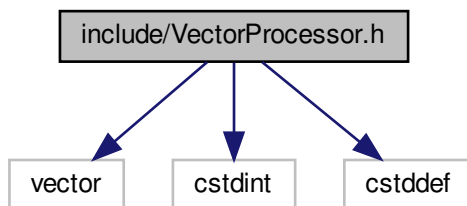
Класс для обработки векторных данных

```

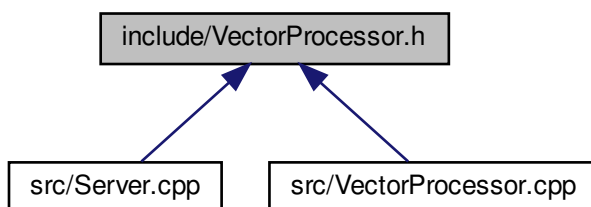
#include <vector>
#include <cstdint>
#include <cstddef>

```

Граф включаемых заголовочных файлов для `VectorProcessor.h`:



Граф файлов, в которые включается этот файл:



Классы

- class `VectorProcessor`
Класс для обработки векторных данных
- struct `VectorProcessor::VectorResult`
Структура результата обработки векторов

6.11.1 Подробное описание

Класс для обработки векторных данных

Автор

Alexander Melekhin

Версия

1.0

Дата

2024

Обеспечивает вычисление суммы элементов векторов с поддержкой больших чисел и переполнения.

6.12 VectorProcessor.h

См. документацию.

```

1
12 #ifndef VECTORPROCESSOR_H
13 #define VECTORPROCESSOR_H
14
15 #include <vector>
16 #include <stdint>
17 #include <cstdint> // Добавлено для size_t
18
26 class VectorProcessor {
27 public:
31     struct VectorResult {
32         uint32_t count;
33         std::vector<double> sums;
34     };
35
41     static VectorResult processVectors(const std::vector<uint8_t>& binaryData);
42
49     static double calculateVectorSum(const std::vector<double>& vector);
50
51 private:
60     static bool readUInt32(const uint8_t* data, size_t& offset, size_t maxSize, uint32_t& value);
61
70     static bool readDouble(const uint8_t* data, size_t& offset, size_t maxSize, double& value);
71 };
72
73 #endif // VECTORPROCESSOR_H

```

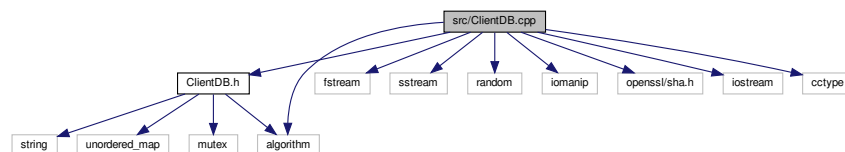
6.13 Файл src/ClientDB.cpp

```

#include "ClientDB.h"
#include <fstream>
#include <sstream>
#include <random>
#include <iomanip>
#include <openssl/sha.h>
#include <iostream>
#include <algorithm>
#include <cctype>

```

Граф включаемых заголовочных файлов для ClientDB.cpp:



6.14 Файл src/Logger.cpp

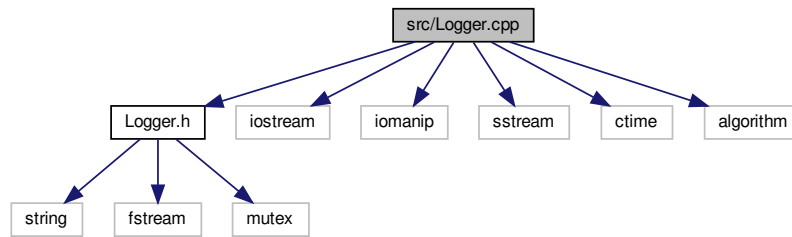
```

#include "Logger.h"
#include <iostream>
#include <iomanip>
#include <sstream>
#include <ctime>

```

```
#include <algorithm>
```

Граф включаемых заголовочных файлов для `Logger.cpp`:



6.15 Файл `src/main.cpp`

```
#include <iostream>
```

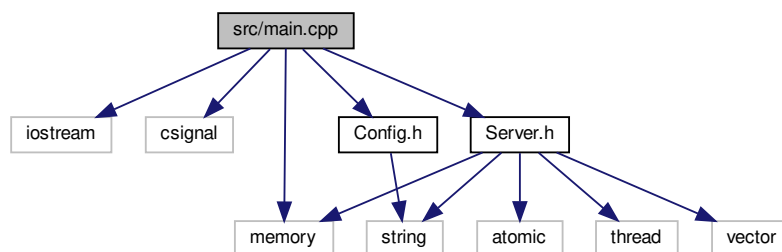
```
#include <csignal>
```

```
#include <memory>
```

```
#include "Server.h"
```

```
#include "Config.h"
```

Граф включаемых заголовочных файлов для `main.cpp`:



Функции

- void `signalHandler` (int signal)
- void `printHelp` ()
- int `main` (int argc, char *argv[])

Переменные

- `std::unique_ptr< Server > server`

6.15.1 Функции

6.15.1.1 main()

```
int main (
    int argc,
    char * argv[] )
```

6.15.1.2 printHelp()

```
void printHelp ( )
```

6.15.1.3 signalHandler()

```
void signalHandler (
    int signal )
```

6.15.2 Переменные

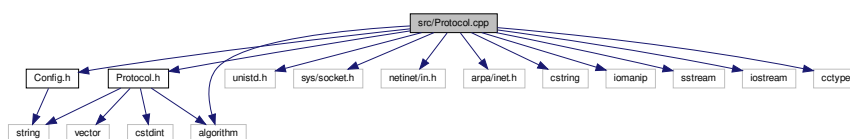
6.15.2.1 server

```
std::unique_ptr<Server> server
```

6.16 Файл src/Protocol.cpp

```
#include "Protocol.h"
#include "Config.h"
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <cstring>
#include <iomanip>
#include <sstream>
#include <iostream>
#include <algorithm>
#include <cctype>
```

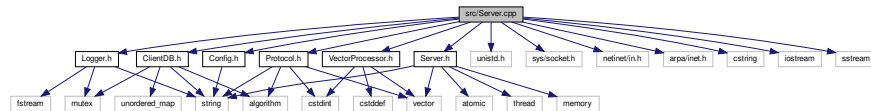
Граф включаемых заголовочных файлов для Protocol.cpp:



6.17 Файл src/Server.cpp

```
#include "Server.h"
#include "Logger.h"
#include "ClientDB.h"
#include "Protocol.h"
#include "VectorProcessor.h"
#include "Config.h"
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <cstring>
#include <iostream>
#include <sstream>
```

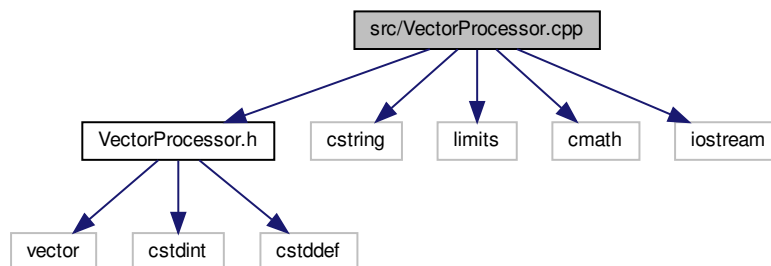
Граф включаемых заголовочных файлов для Server.cpp:



6.18 Файл src/VectorProcessor.cpp

```
#include "VectorProcessor.h"
#include <cstring>
#include <limits>
#include <cmath>
#include <iostream>
```

Граф включаемых заголовочных файлов для VectorProcessor.cpp:



Предметный указатель

- ~Logger
 - Logger, [18](#)
- ~Server
 - Server, [28](#)
- addClient
 - ClientDB, [12](#)
- authenticateClient
 - Server, [28](#)
- binaryToHex
 - Protocol, [21](#)
- BUFFER_SIZE
 - Config, [7](#)
- calculateVectorSum
 - VectorProcessor, [32](#)
- cleanup
 - Server, [28](#)
- ClientDB, [11](#)
 - addClient, [12](#)
 - ClientDB, [12](#)
 - clientExists, [13](#)
 - clients, [16](#)
 - dbMutex, [16](#)
 - filename, [16](#)
 - generateHash, [13](#)
 - generateSalt, [13](#)
 - loadFromFile, [14](#)
 - removeClient, [14](#)
 - saveToFile, [14](#)
 - verifyPassword, [14](#)
- clientDB
 - Server, [30](#)
- clientExists
 - ClientDB, [13](#)
- clients
 - ClientDB, [16](#)
- clientSession
 - Server, [28](#)
- clientThreads
 - Server, [30](#)
- Config, [7](#)
 - BUFFER_SIZE, [7](#)
 - DEFAULT_CLIENT_DB, [7](#)
 - DEFAULT_LOG_FILE, [8](#)
 - DEFAULT_PORT, [8](#)
 - ERR_MSG, [8](#)
 - MAX_LOGIN_LENGTH, [8](#)
 - MAX_PASSWORD_LENGTH, [8](#)
 - OK_MSG, [8](#)
 - SALT_BITS, [9](#)
 - SALT_HEX_LENGTH, [9](#)
- count
 - VectorProcessor::VectorResult, [34](#)
- CRITICAL
 - Logger.h, [42](#)
- dbMutex
 - ClientDB, [16](#)
- DEFAULT_CLIENT_DB
 - Config, [7](#)
- DEFAULT_LOG_FILE
 - Config, [8](#)
- DEFAULT_PORT
 - Config, [8](#)
- ERR_MSG
 - Config, [8](#)
- ERROR
 - Logger.h, [42](#)
- filename
 - ClientDB, [16](#)
 - Logger, [20](#)
- generateHash
 - ClientDB, [13](#)
- generateSalt
 - ClientDB, [13](#)
- getConnectedClients
 - Server, [29](#)
- getCurrentTime
 - Logger, [18](#)
- handleClient
 - Server, [29](#)
- hexToBinary
 - Protocol, [22](#)
- include/ClientDB.h, [37](#), [38](#)
- include/Config.h, [39](#), [40](#)
- include/Logger.h, [41](#), [42](#)
- include/Protocol.h, [43](#), [45](#)
- include/Server.h, [45](#), [47](#)
- include/VectorProcessor.h, [47](#), [49](#)
- INFO
 - Logger.h, [42](#)
- initialize
 - Logger, [18](#)

- Server, [29](#)
- initializeSocket
 - Server, [29](#)
- levelToString
 - Logger, [18](#)
- loadFromFile
 - ClientDB, [14](#)
- log
 - Logger, [19](#)
- logError
 - Logger, [19](#)
- logFile
 - Logger, [20](#)
- Logger, [16](#)
 - ~Logger, [18](#)
 - filename, [20](#)
 - getCurrentTime, [18](#)
 - initialize, [18](#)
 - levelToString, [18](#)
 - log, [19](#)
 - logError, [19](#)
 - logFile, [20](#)
 - Logger, [17, 18](#)
 - logMutex, [20](#)
 - operator=, [19](#)
- logger
 - Server, [31](#)
- Logger.h
 - CRITICAL, [42](#)
 - ERROR, [42](#)
 - INFO, [42](#)
 - LogLevel, [42](#)
 - WARNING, [42](#)
- LogLevel
 - Logger.h, [42](#)
- logMutex
 - Logger, [20](#)
- main
 - main.cpp, [50](#)
- main.cpp
 - main, [50](#)
 - printHelp, [51](#)
 - server, [51](#)
 - signalHandler, [51](#)
- MAX_LOGIN_LENGTH
 - Config, [8](#)
- MAX_PASSWORD_LENGTH
 - Config, [8](#)
- OK_MSG
 - Config, [8](#)
- operator=
 - Logger, [19](#)
- port
 - Server, [31](#)
- printHelp
 - main.cpp, [51](#)
- processVectors
 - VectorProcessor, [33](#)
- Protocol, [20](#)
 - binaryToHex, [21](#)
 - hexToBinary, [22](#)
 - receiveHash, [22](#)
 - receiveLogin, [22](#)
 - receiveVectorData, [23](#)
 - recvAll, [23](#)
 - SEND_RECV_TIMEOUT, [26](#)
 - sendAll, [24](#)
 - sendError, [24](#)
 - sendOk, [25](#)
 - sendSalt, [25](#)
 - sendVectorResults, [25](#)
- readDouble
 - VectorProcessor, [33](#)
- readUInt32
 - VectorProcessor, [33](#)
- receiveHash
 - Protocol, [22](#)
- receiveLogin
 - Protocol, [22](#)
- receiveVectorData
 - Protocol, [23](#)
- recvAll
 - Protocol, [23](#)
- removeClient
 - ClientDB, [14](#)
- running
 - Server, [31](#)
- SALT_BITS
 - Config, [9](#)
- SALT_HEX_LENGTH
 - Config, [9](#)
- saveToFile
 - ClientDB, [14](#)
- SEND_RECV_TIMEOUT
 - Protocol, [26](#)
- sendAll
 - Protocol, [24](#)
- sendError
 - Protocol, [24](#)
- sendOk
 - Protocol, [25](#)
- sendSalt
 - Protocol, [25](#)
- sendVectorResults
 - Protocol, [25](#)
- Server, [26](#)
 - ~Server, [28](#)
 - authenticateClient, [28](#)
 - cleanup, [28](#)
 - clientDB, [30](#)
 - clientSession, [28](#)
 - clientThreads, [30](#)

- getConnectedClients, [29](#)
- handleClient, [29](#)
- initialize, [29](#)
- initializeSocket, [29](#)
- logger, [31](#)
- port, [31](#)
- running, [31](#)
- Server, [27](#)
- serverSocket, [31](#)
- start, [30](#)
- stop, [30](#)
- waitForStop, [30](#)
- server
 - main.cpp, [51](#)
- serverSocket
 - Server, [31](#)
- signalHandler
 - main.cpp, [51](#)
- src/ClientDB.cpp, [49](#)
- src/Logger.cpp, [49](#)
- src/main.cpp, [50](#)
- src/Protocol.cpp, [51](#)
- src/Server.cpp, [52](#)
- src/VectorProcessor.cpp, [52](#)
- start
 - Server, [30](#)
- stop
 - Server, [30](#)
- sums
 - VectorProcessor::VectorResult, [35](#)
- VectorProcessor, [31](#)
 - calculateVectorSum, [32](#)
 - processVectors, [33](#)
 - readDouble, [33](#)
 - readUInt32, [33](#)
- VectorProcessor::VectorResult, [34](#)
 - count, [34](#)
 - sums, [35](#)
- verifyPassword
 - ClientDB, [14](#)
- waitForStop
 - Server, [30](#)
- WARNING
 - Logger.h, [42](#)